

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	V.Lasya	Reg. No.:	192524186
Section / Batch:	B.Tech AI & DS	Date:	17-07-2026

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – Technical Presentation

Q1. Visualization-Based Technical Presentation on Growth Functions

Analyze growth functions $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$ using visualization tools in a technical presentation format. Clearly define the purpose of comparing these functions for understanding algorithm efficiency. Use graphs, tables, or charts to show how each function behaves as the input size increases. Explain which functions remain practical for large-scale problems and justify your reasoning. Present the conclusions clearly and professionally by linking growth behavior to algorithm selection.

Growth Functions, Visualized

Comparing $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ and $O(2^n)$ to guide algorithm selection at scale

$O(\log n)$

$O(n)$

$O(n \log n)$

$O(n^2)$

$O(2^n)$

Why Compare Growth Functions?

Big-O notation strips away hardware and implementation noise, leaving the one property that determines whether an algorithm survives contact with real-world data: how its cost grows as input size increases.

1

Predict scalability

Estimate whether an algorithm will still run in reasonable time as n grows from thousands to billions.

2

Compare fairly

Judge algorithms on their own logic, independent of CPU speed, language, or hardware generation.

3

Spot performance cliffs

Identify the input size at which an algorithm shifts from usable to impractical.

4

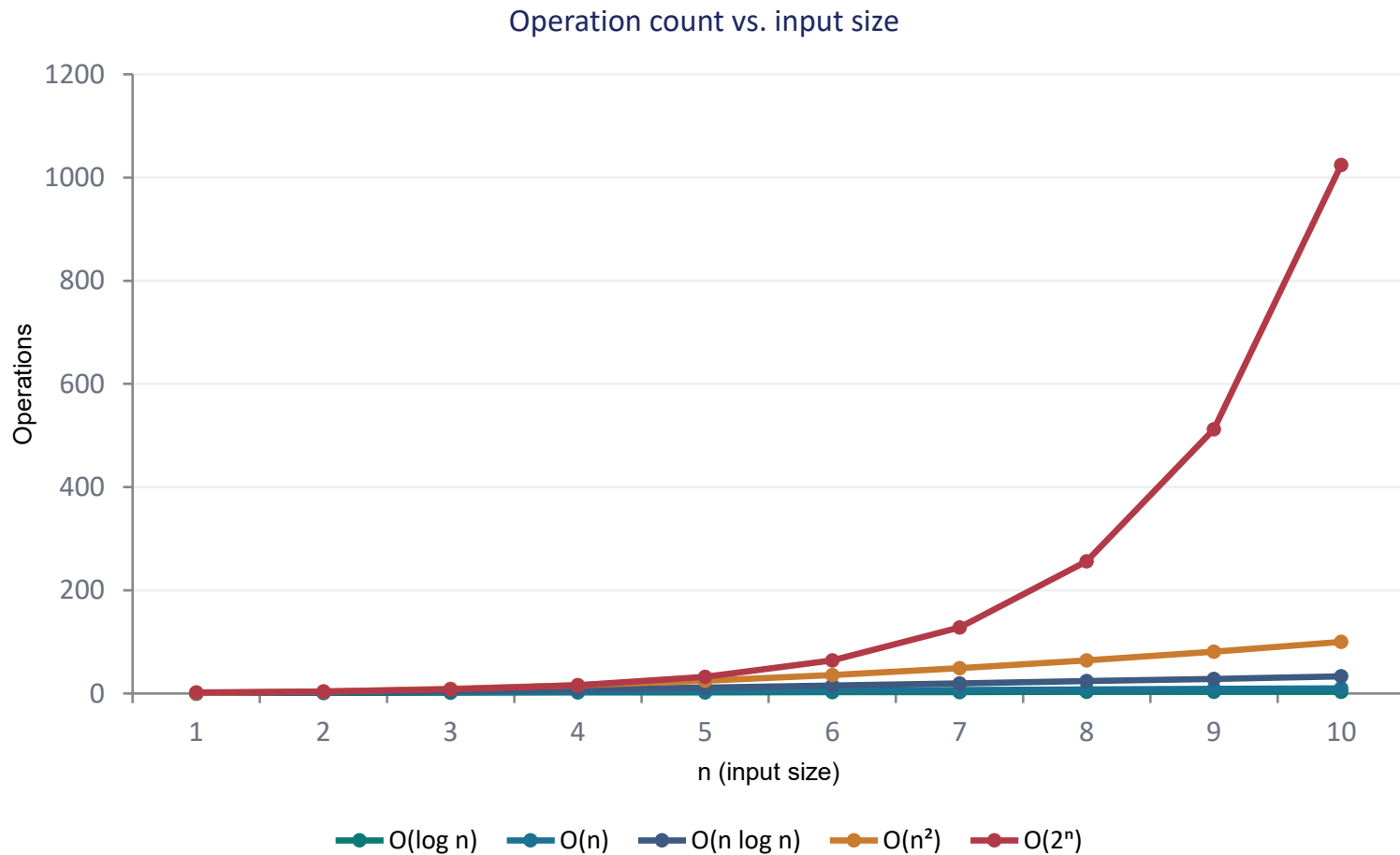
Guide design decisions

Choose the right algorithm and data structure before implementation, not after a production slowdown.

The Five Growth Functions

Notation	Name	Behavior as n grows	Typical Example
$O(\log n)$	Logarithmic	Cost barely increases — each step discards half the remaining input.	<i>Binary Search</i>
$O(n)$	Linear	Cost increases in direct proportion to input size.	<i>Linear Search / Array Scan</i>
$O(n \log n)$	Linearithmic	Cost grows slightly faster than linear — the practical ceiling for sorting.	<i>Merge Sort / Quick Sort</i>
$O(n^2)$	Quadratic	Cost grows with the square of input size — doubling n quadruples cost.	<i>Bubble Sort / Insertion Sort</i>
$O(2^n)$	Exponential	Cost doubles with every additional element — explodes almost immediately.	<i>Brute-force Subset Sum</i>

Growth Trends for Small Inputs ($n = 1-10$)



Even across just ten inputs, $O(2^n)$ has already pulled far ahead of every other curve — a signal of what happens as n keeps growing.

At $n = 10$:

$O(2^n) = 1,024$ ops

$O(n^2) = 100$ ops

$O(n \log n) \approx 80$ ops

$O(n) = 10$ ops

$O(\log n) \approx 3$ ops

Growth Behavior at Real-World Scale

Function	n = 10	n = 100	n = 1,000	n = 1,000,000
$O(\log n)$	3	7	10	20
$O(n)$	10	100	1,000	1,000,000
$O(n \log n)$	33	664	9,966	$\approx 19,900,000$
$O(n^2)$	100	10,000	1,000,000	1,000,000,000,000
$O(2^n)$	1,024	$\approx 1.27 \times 10^{30}$	astronomically large	incomputable

Shaded cells mark the point at which a function stops being practical. $O(n^2)$ is already at one trillion operations by $n = 1,000,000$; $O(2^n)$ passes the number of atoms in the observable universe before n reaches 300.

What Remains Practical at Large Scale?

Scales Well

$O(\log n) \cdot O(n) \cdot O(n \log n)$

Stay fast even as n reaches billions.
These are the growth classes production systems are built on.

Use With Caution

$O(n^2)$

Acceptable for small or medium datasets (roughly $n < 10,000$), but costly beyond that — evaluate before deploying at scale.

Avoid at Scale

$O(2^n)$

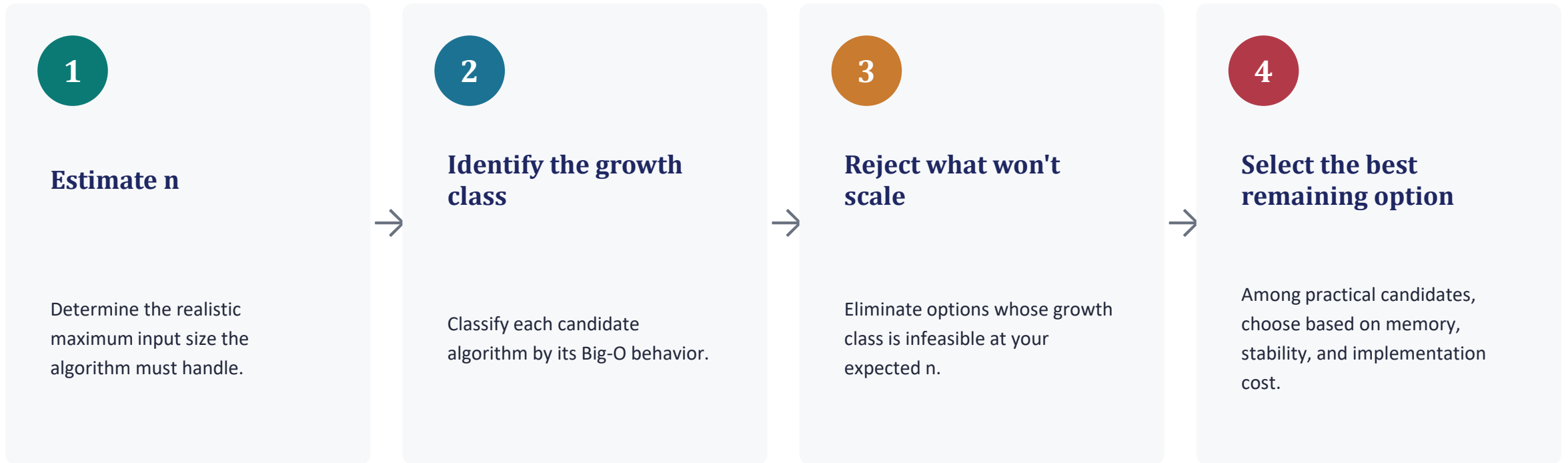
Becomes infeasible past $n \approx 40$ –50.
Reserve for small, bounded, or last-resort brute-force cases.

Linking Growth Behavior to Algorithm Choice

Problem Type	Target Growth	Example Algorithm
Searching sorted data	$O(\log n)$	<i>Binary Search</i>
Processing every record once	$O(n)$	<i>Linear scan, hashing lookups</i>
Sorting large datasets	$O(n \log n)$	<i>Merge Sort, Quick Sort, Heap Sort</i>
Simple sort on small n (< 50)	$O(n^2)$ acceptable	<i>Insertion Sort</i>
Exhaustive / combinatorial search	$O(2^n)$ only if n is tiny	<i>Brute-force subset enumeration</i>

The growth class of a candidate algorithm should be checked against the expected input size before it is chosen — not after performance problems appear in production.

A Simple Decision Path



CONCLUSION

Growth Rate Determines Algorithm Selection



Growth rate — not raw speed on today's hardware — determines whether an algorithm scales into the future.



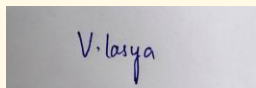
$O(\log n)$, $O(n)$, and $O(n \log n)$ should be preferred whenever a suitable algorithm exists in that class.



$O(n^2)$ and $O(2^n)$ must be reserved for small, bounded inputs, or replaced before they reach production scale.

Code of Conduct

I V.Lasya(192524186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1	4	4	4	2	3	90
Overall Total (100)						90

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

Date: _____

Date: _____

Date: _____